プロジェクト管理API仕様書

作成日: 2026-01-21 16:30
最終更新: 2026-03-12 15:22
バージョン: 3.0.0
対象画面: 実験管理者 - プロジェクト管理画面、実験作業者画面
総API数: 20個
実装状況: 18個実装済み、1個未実装、1個廃止済み

📊 実装状況サマリー

カテゴリ	実装済み	未実装	廃止済み	合計
基本CRUD	2	0	0	2
チェックイン/チェックアウト	3	0	0	3
データ取得	2	0	0	2
指示管理	4	0	0	4
プロジェクト検索	0	1	0	1
実験作業者用	4	0	0	4
実験データ	2	0	1	3
合計	**18**	**1**	**1**	**20**

完了率: 90% (18/20)

🎯 API一覧

#	API名	メソッド	エンドポイント	状態	優先度
1	プロジェクト一覧取得	GET	`/api/v1/projects`	✅ 実装済み	-
2	プロジェクト作成	POST	`/api/v1/projects`	✅ 実装済み	-
3	チェックアウト	POST	`/api/v1/projects/:projectId/checkout`	✅ 実装済み	-
4	チェックイン	POST	`/api/v1/projects/:projectId/checkin`	✅ 実装済み	-
5	ファイル一覧取得	GET	`/api/v1/projects/:projectName/files`	✅ 実装済み	-
6	タスク一覧取得	GET	`/api/v1/projects/:projectName/tasks`	✅ 実装済み	-
7	プロジェクト詳細取得	GET	`/api/v1/projects/:projectId`	✅ 実装済み	-
8	チェックアウト状態取得	GET	`/api/v1/projects/:projectId/checkout-status`	✅ 実装済み	-
9	指示一覧取得（実験作業者用）	GET	`/api/v1/projects/:projectName/worker-instructions`	✅ 実装済み（`getWorkerInstructions`）	-
10	指示確認完了	PUT	`/api/v1/projects/:projectName/instructions/:instructionId/confirm`	✅ 実装済み（`confirmInstruction`）	-

11	承認依頼	PATCH	`/api/v1/projects/:projectName/approval-request`	✅ 実装済み	-
12	実験作業者用プロジェクト一覧	GET	`/api/v1/projects/worker-accessible`	✅ 実装済み	-
13	MAIMLファイル取得（プロジェクト）	GET	`/api/v1/projects/:projectName/maiml`	✅ 実装済み	-
14	指示送信（タスク作成）	POST	`/api/v1/projects/:projectName/instructions`	🔴 未実装	⭐⭐⭐ 高
15	指示一覧取得（実験管理者用）	GET	`/api/v1/projects/:projectName/instructions`	🔴 未実装	⭐⭐ 中
16	プロジェクト詳細検索	POST	`/api/v1/projects/search`	🔴 未実装	⭐ 低
17	タスク登録情報取得	GET	`/api/v1/projects/:projectName/task-registration`	✅ 実装済み	⭐⭐⭐ 高
18	タスク登録保存	POST	`/api/v1/projects/:projectName/task-registration`	✅ 実装済み	⭐⭐⭐ 高
21	テンプレート実績取得	GET	`/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data`	✅ 実装済み	⭐⭐⭐ 高
22	実験データー括保存	POST	`/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data`	✅ 実装済み	⭐⭐⭐ 高
23	トランジションステータス更新（廃止済み）	PUT	`/api/v1/experiments/projects/:projectId/methods/:methodId/transitions/:transitionId/status`	⛔ 廃止済み（410）	-

🔐 共通仕様

認証・認可

認証方式: JWT + Session Hybrid

必須ヘッダー:

```
Authorization: Bearer maiml_session-{sessionId}_{timestamp}
```

organization_id の取得:

- ログイン時に `X-Organization-Id` ヘッダーから組織IDを取得し、セッションに保存します
- ログイン後のAPIでは、 `Authorization` ヘッダーのセッショントークンから自動的に取得されます
- リクエストヘッダーで `X-Organization-Id` を指定する必要はありません
- `authMiddleware` がセッション情報を検証し、ユーザー情報（organization_idを含む）を `req.user` に格納します

権限要件:

- すべてのAPI: `experiment_admin` または `system_admin` ロール必須
- `experiment_worker` ロールは403 Forbiddenを返却

ミドルウェアチェーン:

```
router.METHOD('/path',
  authMiddleware,           // 1. JWT認証（セッション検証、req.userに格納）
  permissionCheck([...]),   // 2. 権限チェック
  controller.handler        // 3. コントローラー処理
);
```

ミドルウェアの動作:

1. `authMiddleware`: JWTトークン（`maiml_session-{sessionId}_{timestamp}`）を検証し、セッション情報を取得して`req.user`に格納（organizationIdを含む）
2. `permissionCheck`: ユーザーのロール（`req.user.role`）を検証
3. コントローラー: `req.user.organizationId`を使用してテナント分離されたデータにアクセス

マルチテナント分離

- すべてのDBクエリに`organization_id`フィルタ必須
- すべてのS3パスに`org_{organization_prefix}`プレフィックス必須
- クロステナントアクセスは厳禁（System Admin除く）

統一レスポンス形式

成功レスポンス:

```
{
  "success": true,
  "data": { ... },
  "pagination": {  // オプショナル
    "page": 1,
    "limit": 20,
    "totalItems": 150,
    "totalPages": 8,
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラーレスポンス:

```
{
  "success": false,
  "error": {
    "code": "ERROR_CODE",
    "message": "エラーメッセージ",
    "details": [
      {
        "field": "fieldName",
        "message": "詳細メッセージ",
        "code": "FIELD_ERROR_CODE"
```

```
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

共通エラーコード

コード	HTTPステータス	説明
`AUTH_TOKEN_INVALID`	401	認証トークンが無効
`PERMISSION_INSUFFICIENT`	403	権限不足
`RESOURCE_NOT_FOUND`	404	リソースが見つからない
`VALIDATION_ERROR`	400	バリデーションエラー
`VALIDATION_REQUIRED_FIELD`	400	必須フィールドが未入力
`VALIDATION_INVALID_FORMAT`	400	フォーマットが不正
`SYSTEM_ERROR`	500	システムエラー
`SYSTEM_DATABASE_ERROR`	500	データベースエラー

📋 API詳細仕様

1. プロジェクト一覧取得 ✅

エンドポイント: `GET /api/v1/projects`

説明: 組織内のプロジェクト一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

リクエストパラメータ

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`page`	integer	×	1	ページ番号（1以上）
`limit`	integer	×	20	1ページあたりの件数（1-100）
`include_completed`	boolean	×	false	完了プロジェクトを含める
`sort_by`	string	×	`create_at`	ソート項目（`create_at`, `project_name`）
`order`	string	×	`desc`	ソート順（`asc`, `desc`）

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": [
    {
      "project_id": "550e8400-e29b-41d4-a716-446655440000",
      "project_name": "Sample Project",
      "description": "プロジェクトの説明",
      "project_status": "created",
      "is_checked_out": false,
      "checked_out_by": null,
      "checked_out_at": null,
      "checked_out_user_name": null,
      "create_at": "2026-01-21T10:00:00Z",
      "update_at": "2026-01-21T10:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 150,
    "totalPages": 8,
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 401: 認証エラー
- 403: 権限不足
- 400: バリデーションエラー
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts` (`list`)
- **サービス**: `backend/src/services/projectService.ts` (`list`)

---

2. プロジェクト作成 ✅

エンドポイント: `POST /api/v1/projects`

説明: 新規プロジェクトを作成

認証: 必須

権限: `experiment_admin`, `system_admin`

リクエスト

ボディ（JSON）:

```
{
  "projectName": "New Project",
  "description": "プロジェクトの説明（オプション）"
}
```

バリデーション:

フィールド	型	必須	制約
`projectName`	string	○	1-255文字、制御文字不可
`description`	string	×	0-1000文字

レスポンス

成功時 (201 Created):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "New Project",
    "description": "プロジェクトの説明",
    "project_status": "created",
    "organization_id": "org-uuid",
    "department_id": "dept-uuid",
    "create_user_id": "user-uuid",
    "create_at": "2026-01-21T16:00:00Z",
    "update_at": "2026-01-21T16:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 409: プロジェクト名重複（`BUSINESS_DUPLICATE_PROJECT_NAME`）
- 401: 認証エラー
- 403: 権限不足
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`create`）
- **サービス**: `backend/src/services/projectService.ts`（`create`）

---

3. チェックアウト ✅

エンドポイント: `POST /api/v1/projects/:projectName/checkout`

説明: プロジェクトをチェックアウトし、排他編集を開始

認証: 必須

権限: `experiment_admin`, `system_admin`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "is_checked_out": true,
    "checked_out_by": "user-uuid",
    "checked_out_at": "2026-01-21T16:00:00Z",
    "project_status": "in_progress"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、プロジェクトステータス不正
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない
- 409: 既にチェックアウト済み（`PROJECT_ALREADY_CHECKED_OUT`）
- 500: システムエラー

ビジネスルール

- プロジェクトステータスが `created` または `checked_in` の場合のみチェックアウト可能
- チェックアウト中は他のユーザーは編集不可
- チェックアウト時に指示が存在しない場合はエラー

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`checkout`）
- **サービス**: `backend/src/services/projectService.ts`（`checkout`）

---

4. チェックイン ✅

エンドポイント: `POST /api/v1/projects/:projectName/checkin`

説明: プロジェクトをチェックインし、排他編集を終了

認証: 必須

権限: `experiment_admin`, `system_admin`

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "is_checked_out": false,
    "checked_out_by": null,
    "checked_out_at": null,
    "project_status": "checked_in"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: 実行中のタスクあり（`TASK_IN_PROGRESS`）
- 401: 認証エラー
- 403: 権限不足（他ユーザーがチェックアウト中）
- 404: プロジェクトが存在しない
- 500: システムエラー

ビジネスルール

- チェックアウトしたユーザーのみチェックイン可能
- 実行中のタスクがある場合はチェックイン不可
- チェックイン時にプロジェクトステータスを `checked_in` に更新

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`checkin`）
- **サービス**: `backend/src/services/projectService.ts`（`checkin`）

---

5. ファイル一覧取得 ✅

エンドポイント: `GET /api/v1/projects/:projectName/files`

説明: プロジェクト内のファイル一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`include_archive`	boolean	×	false	アーカイブファイルを含める

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "files": [
      {
        "file_id": "file-uuid",
        "file_name": "sample_project.maiml",
        "file_path": "org_maiml_lab_001/projects/Sample_Project/sample_project.maiml",
        "file_size": 12345,
        "file_type": "maiml",
        "is_archive": false,
```

```
        "version": null,
        "created_at": "2026-01-21T10:00:00Z",
        "updated_at": "2026-01-21T16:00:00Z"
      }
    ],
    "archive_files": [
      {
        "file_id": "archive-file-uuid",
        "file_name": "sample_project_v1.maiml",
        "file_path": "org_maiml_lab_001/projects/Sample_Project/archive/sample_project_v1.maiml",
        "file_size": 11234,
        "file_type": "maiml",
        "is_archive": true,
        "version": 1,
        "created_at": "2026-01-20T10:00:00Z",
        "updated_at": "2026-01-20T10:00:00Z"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`getProjectFiles`）
- **サービス**: `backend/src/services/projectService.ts`（`getProjectFiles`）

実装要件

- S3から `org_{organization_prefix}/projects/{project_name}/` 配下のファイル一覧を取得
- archiveファイルは常に取得される（`include_archive` パラメータは削除済み）
- バージョン番号は `_v{number}` から抽出
- 署名付きURL付きで返却

---

6. タスク一覧取得 ✅

エンドポイント: `GET /api/v1/projects/:projectName/tasks`

説明: プロジェクト内のタスク一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`status`	string	×	all	タスクステータスフィルタ（`all`, `pending`, `in_progress`, `completed`, `failed`）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
    "tasks": [
      {
        "task_id": "task-uuid",
        "task_name": "実験タスク1",
        "task_status": "in_progress",
        "assigned_to": "worker-uuid",
        "assigned_user_name": "Worker Name",
        "method_name": "Method 1",
        "transition_name": "Transition A",
        "created_at": "2026-01-21T10:00:00Z",
        "updated_at": "2026-01-21T16:00:00Z",
        "completed_at": null
      }
    ],
    "statistics": {
      "total": 10,
      "pending": 2,
      "in_progress": 5,
      "completed": 3,
      "failed": 0
    }
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足

- 404: プロジェクトが存在しない
- 500: システムエラー

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`getProjectTasks`）
- **サービス**: `backend/src/services/projectService.ts`（`getProjectTasks`）

実装要件

- `project_task` テーブルから取得
- ステータスフィルタリング対応
- ワーカー名、メソッド名、トランジション名をJOINで取得
- タスク統計を集計

---

7. 指示送信（タスク作成） 🔴

エンドポイント: `POST /api/v1/projects/:projectName/instructions`

説明: プロジェクトに対して実験指示を送信し、タスクを作成

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

ボディ（JSON）:

```
{
  "instructionTitle": "実験指示タイトル",
  "instructionContent": "実験指示の詳細内容",
  "dueDate": "2026-01-31T23:59:59Z",
  "tasks": [
    {
      "methodId": "method-uuid",
      "transitionId": "transition-uuid",
      "assignedTo": "worker-uuid",
      "priority": "high"
    }
  ]
}
```

バリデーション:

フィールド	型	必須	制約
`instructionTitle`	string	○	1-255文字
`instructionContent`	string	○	1-5000文字
`dueDate`	ISO8601	×	未来の日時
`tasks`	array	○	1個以上
`tasks[].methodId`	UUID	○	有効なメソッドID
`tasks[].transitionId`	UUID	○	有効なトランジションID
`tasks[].assignedTo`	UUID	○	有効なワーカーID
`tasks[].priority`	string	×	`low`, `medium`, `high`

レスポンス

成功時 (201 Created):

```
{
  "success": true,
  "data": {
    "instruction_id": "instruction-uuid",
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "instruction_title": "実験指示タイトル",
    "instruction_content": "実験指示の詳細内容",
    "due_date": "2026-01-31T23:59:59Z",
    "created_tasks": [
      {
        "task_id": "task-uuid",
        "method_id": "method-uuid",
        "transition_id": "transition-uuid",
        "assigned_to": "worker-uuid",
        "task_status": "pending"
      }
    ],
    "created_at": "2026-01-21T16:00:00Z"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、期限が過去
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクト、メソッド、トランジション、ワーカーが存在しない
- 500: システムエラー

実装要件

- `project_instruction` テーブルに指示を登録

- 各タスクを `project_task` テーブルに登録
- トランザクション処理必須

---

8. 指示一覧取得 🔴

エンドポイント: `GET /api/v1/projects/:projectName/instructions`

説明: プロジェクトの指示一覧を取得

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`page`	integer	×	1	ページ番号
`limit`	integer	×	20	1ページあたりの件数（1-100）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": [
    {
      "instruction_id": "instruction-uuid",
      "instruction_title": "実験指示タイトル",
      "instruction_content": "実験指示の詳細内容",
      "due_date": "2026-01-31T23:59:59Z",
      "task_count": 5,
      "completed_tasks": 2,
      "created_at": "2026-01-21T10:00:00Z",
      "created_by": "user-uuid",
      "created_by_name": "Manager Name"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 50,
    "totalPages": 3,
    "hasNext": true,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
```

```
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない
- 500: システムエラー

---

9. プロジェクト詳細検索 ✅

エンドポイント: `POST /api/v1/projects/search`

注意: 実装ファイルではルートが定義されていますが、コントローラー/サービス層の実装状況を要確認

説明: 複雑な条件でプロジェクトを検索

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエスト

ボディ（JSON）:

```
{
  "projectName": "検索キーワード",
  "status": ["created", "in_progress"],
  "dateRange": {
    "start": "2026-01-01",
    "end": "2026-01-31"
  },
  "createdBy": "user-uuid",
  "hasActiveTasks": true,
  "page": 1,
  "limit": 20
}
```

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": [
    {
      "project_id": "550e8400-e29b-41d4-a716-446655440000",
      "project_name": "Sample Project",
      "description": "プロジェクトの説明",
      "project_status": "in_progress",
```

```
      "is_checked_out": true,
      "checked_out_by": "user-uuid",
      "active_tasks_count": 5,
      "create_at": "2026-01-15T10:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 10,
    "totalPages": 1,
    "hasNext": false,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

---

10. プロジェクト公開 ✅

エンドポイント: `POST /api/v1/projects/:projectName/publish`

注意: 実装ファイルではルートが定義されていますが、コントローラー/サービス層の実装状況を要確認

説明: プロジェクトを公開状態に変更

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

ボディ（JSON）:

```
{
  "publishNote": "公開コメント（オプション）"
}
```

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "550e8400-e29b-41d4-a716-446655440000",
    "project_name": "Sample Project",
```

```
    "project_status": "published",
    "published_at": "2026-01-21T16:00:00Z",
    "published_by": "user-uuid"
  },
  "meta": {
    "timestamp": "2026-01-21T16:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: プロジェクトステータスが不正
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない
- 500: システムエラー

ビジネスルール

- 完了済みのプロジェクトのみ公開可能
- 公開後は編集不可

---

11. 指示一覧取得（実験作業者用） ✅

エンドポイント: `GET /api/v1/projects/:projectName/worker-instructions`

説明: 実験作業者が自分に割り当てられた指示一覧を取得

認証: 必須

権限: `experiment_worker`（読取専用）

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`projectId`	UUID	×	-	プロジェクトIDでフィルタリング

レスポンス

成功時 (200 OK):

```
{
  "success": true,
```

```
  "data": [
    {
      "instruction_id": "instruction-uuid",
      "project_id": "project-uuid",
      "project_name": "Sample Project",
      "method_id": "method1",
      "method_name": "合成実験",
      "task_no": 1,
      "transition_id": "transition-uuid-1",
      "transition_name": "材料準備トランジション",
      "instruction_target": "task",
      "instruction_content": "指示内容の詳細",
      "priority": "high",
      "type": "normal",
      "is_acknowledged": false,
      "acknowledged_at": null,
      "acknowledged_user_id": null,
      "created_at": "2026-01-21T10:00:00Z",
      "created_by": "manager-uuid",
      "created_by_name": "Manager Name"
    },
    {
      "instruction_id": "instruction-uuid-2",
      "project_id": "project-uuid",
      "project_name": "Sample Project",
      "method_id": "method1",
      "method_name": "合成実験",
      "task_no": 2,
      "transition_id": "transition-uuid-2",
      "transition_name": "反応実験トランジション",
      "instruction_target": "transition",
      "instruction_content": "トランジション指示の詳細",
      "priority": "medium",
      "type": "urgent",
      "is_acknowledged": true,
      "acknowledged_at": "2026-01-21T14:00:00Z",
      "acknowledged_user_id": "worker-uuid",
      "created_at": "2026-01-21T10:00:00Z",
      "created_by": "manager-uuid",
      "created_by_name": "Manager Name"
    }
  ],
  "meta": {
    "timestamp": "2026-01-27T19:00:00Z",
    "requestId": "req_abc123"
  }
}
```


エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 500: システムエラー

実装要件

DBクエリ:

```
// project_instructionテーブルから指示を取得
let query = db
```

```
  .selectFrom('project_instruction')
  .innerJoin('project', 'project_instruction.project_id', 'project.project_id')
  .leftJoin('organization_user as creator', 'project_instruction.create_user_id', 'creator.user_id')
  .leftJoin('user_authentication', 'creator.user_id', 'user_authentication.user_id')
  .where('project_instruction.organization_id', '=', organizationId);

// 実験作業者の場合は自分が担当するタスクに紐づく指示のみ
if (req.user.role === 'experiment_worker') {
  query = query
    .innerJoin('project_task', 'project_instruction.instruction_id', 'project_task.instruction_id')
    .where('project_task.assign_user_id', '=', req.user.userId);
}

// フィルタリング
if (projectId) {
  query = query.where('project_instruction.project_id', '=', projectId);
}

const instructions = await query
  .select([
    'project_instruction.*',
    'project.project_name',
    'user_authentication.user_name as created_by_name'
  ])
  .orderBy('project_instruction.create_at', 'desc')
  .execute();
```

メソッド名・タスク名の取得:

```
// MAIMLファイルからメソッド情報を取得
for (const instruction of instructions) {
  const project = await db
    .selectFrom('project')
    .where('project_id', '=', instruction.project_id)
    .select('project_name')
    .executeTakeFirst();

  if (project) {
    const maimlContent = await storageService.downloadFile(
      organizationPrefix,
      `projects/${project.project_name}/${project.project_name}.maiml`
    );

    const methods = await parseMultipleMethodsStructure(maimlContent);
    const method = methods.find(m => m.methodId === instruction.method_id);

    instruction.method_name = method?.methodName || '';

    if (instruction.task_no && method) {
      const task = method.tasks?.find(t => t.task_no === instruction.task_no);
      instruction.task_name = task?.task_name || '';
    }
  }
}
```

---

12. 指示確認完了 🔴

エンドポイント: `PUT /api/v1/projects/:projectName/instructions/:instructionId/confirm`

説明: 指示を確認済みに変更

認証: 必須

権限: `experiment_worker`（書込可）

優先度: ⭐⭐ 中

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名
`instructionId`	UUID	○	指示ID

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "instruction_id": "instruction-uuid",
    "is_acknowledged": true,
    "acknowledged_at": "2026-01-27T19:00:00Z",
    "acknowledged_user_id": "worker-uuid"
  },
  "meta": {
    "timestamp": "2026-01-27T19:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、既に確認済み
- 401: 認証エラー
- 403: 権限不足（担当者以外が確認しようとした）
- 404: 指示が存在しない
- 500: システムエラー

ビジネスロジック

前提条件チェック:

```
// 指示が存在するか確認
const instruction = await db
  .selectFrom('project_instruction')
  .where('instruction_id', '=', instructionId)
  .where('organization_id', '=', organizationId)
  .selectAll()
  .executeTakeFirstOrThrow();

// 実験作業者の場合は自分のタスクに紐づく指示のみ
if (req.user.role === 'experiment_worker') {
  const task = await db
```

```
    .selectFrom('project_task')
    .where('instruction_id', '=', instructionId)
    .where('assign_user_id', '=', req.user.userId)
    .where('organization_id', '=', organizationId)
    .executeTakeFirst();

  if (!task) {
    throw new ForbiddenError('自分に割り当てられていない指示は確認できません');
  }
}

// 既に確認済みの場合はエラー
if (instruction.is_acknowledged) {
  throw new BadRequestError('この指示は既に確認済みです');
}
```

実装要件

DBクエリ:

```
// 指示を確認済みに更新
await db
  .updateTable('project_instruction')
  .set({
    is_acknowledged: true,
    acknowledged_at: new Date().toISOString(),
    acknowledged_user_id: req.user.userId,
    update_at: new Date().toISOString()
  })
  .where('instruction_id', '=', instructionId)
  .where('organization_id', '=', organizationId)
  .execute();

// 更新後のユーザー名を取得
const user = await db
  .selectFrom('organization_user')
  .innerJoin('user_authentication', 'organization_user.user_id', 'user_authentication.user_id')
  .where('organization_user.user_id', '=', req.user.userId)
  .select('user_authentication.user_name')
  .executeTakeFirst();
```

---

13. 承認依頼 ✅

エンドポイント: `PATCH /api/v1/projects/:projectName/approval-request`

説明: 実験作業完了後に実験管理者へ承認依頼を送信。プロジェクトのステータスを承認待ち（3）に変更する。

認証: 必須

権限: `experiment_worker`（書込可、機能CD: `050000`）

ミドルウェア: `authMiddleware` → `tenantIsolation` → `checkPermission('050000', 1)` → バリデーション

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明

`projectName`	string	○	プロジェクト名

ボディ（JSON）:

```
{
  "comment": "すべての実験が完了しました。ご確認をお願いします。"
}
```

バリデーション:

フィールド	型	必須	制約
`comment`	string	×	0-1000文字

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "project_id": "project-uuid",
    "project_name": "Sample Project",
    "previous_status": "active",
    "new_status": "waiting_approval",
    "requested_by": "worker-uuid",
    "requested_by_name": "Worker Name",
    "requested_at": "2026-01-27T19:00:00Z",
    "comment": "すべての実験が完了しました。ご確認をお願いします。"
  },
  "meta": {
    "timestamp": "2026-01-27T19:00:00Z",
    "requestId": "req_abc123"
  }
}
```

注意: レスポンスに `notification_id` は含まれません。

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 404: プロジェクトが存在しない
- 409: プロジェクトステータスが不正（`BUSINESS_STATUS_INVALID`）
- 500: システムエラー（`SYSTEM_DATABASE_ERROR`）

ビジネスロジック

前提条件チェック:

- プロジェクトの `project_status` が `2`（タスク実行中）であることを確認
- 条件を満たさない場合は `BUSINESS_STATUS_INVALID` (409) を返却
- タスク完了チェック・未解決問題チェックは**実装なし**

処理内容:

- `project_status` を `3`（承認待ち）に更新
- `project_task_detail` の `task_status` が `1`（IN_PROGRESS: 実行中）のレコードを `2`（REVIEW_WAITING: レビュー待ち）に一括更新（同一トランザクション内）
- `previous_status` は `"active"`（ハードコード）
- `new_status` は `"waiting_approval"`（ハードコード）

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts`（`requestApproval`）
- **サービス**: `backend/src/services/projectService.ts`（`requestApproval`）
- **リポジトリ**: `backend/src/repositories/projectRepository.ts`（`findProjectIdByName`, `updateApprovalStatus`）
- **リポジトリ**: `backend/src/repositories/projectTaskRepository.ts`（`bulkUpdateTaskStatusByProject`）

---

14. 実験作業者用プロジェクト一覧取得 ✅

エンドポイント: `GET /api/v1/projects/worker-accessible`

説明: 実験作業者が選択可能なプロジェクト一覧を取得

認証: 必須

権限: `experiment_worker`（読取専用、機能CD: `050000`）

ミドルウェア: `authMiddleware` → `tenantIsolation` → `checkPermission('050000', 0)`

リクエストパラメータ

クエリパラメータ:

パラメータ	型	必須	デフォルト	説明
`page`	integer	×	1	ページ番号（1以上）
`limit`	integer	×	20	1ページあたりの件数（1-100）
`status`	string	×	-	ステータスフィルタ（`"1"`, `"2"`, `"3"` の数値文字列）
`sort_by`	string	×	`create_at`	ソート項目（`create_at`, `project_name`, `update_at`）
`order`	string	×	`asc`	ソート順（`asc`, `desc`）

注意: `status` は数値文字列で指定します。`"1"`, `"2"`, `"3"` などのプロジェクトステータスコードに対応します。

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": [
    {
      "projectId": "project-uuid",
      "projectName": "Sample Project",
      "description": "プロジェクトの説明",
      "projectStatus": 2,
      "useDeemedProgress": false,
      "createdAt": "2026-01-21T10:00:00Z",
      "updatedAt": "2026-01-27T19:00:00Z"
    },
    {
      "projectId": "project-uuid-2",
      "projectName": "Another Project",
      "description": "別のプロジェクト",
      "projectStatus": 3,
      "useDeemedProgress": true,
      "createdAt": "2026-01-20T10:00:00Z",
      "updatedAt": "2026-01-27T18:00:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "totalItems": 15,
    "totalPages": 1,
    "hasNext": false,
    "hasPrev": false
  },
  "meta": {
    "timestamp": "2026-01-27T19:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンスフィールド説明:

フィールド	型	説明
`projectId`	string (UUID)	プロジェクトID
`projectName`	string	プロジェクト名
`description`	string	プロジェクト説明
`projectStatus`	number	プロジェクトステータス（数値）
`useDeemedProgress`	boolean	みなし進捗使用フラグ
`createdAt`	string (ISO 8601)	作成日時
`updatedAt`	string (ISO 8601)	更新日時

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 500: システムエラー

ビジネスロジック

- 自組織内のプロジェクトのみ返却（マルチテナント分離）
- `experiment_worker` ユーザーは自分がタスク担当として登録されているプロジェクトのみ参照可能

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/workerProjectController.ts`（`listWorkerAccessible`）
- **サービス**: `backend/src/services/workerProjectService.ts`（`listWorkerAccessible`）
- **リポジトリ**: `backend/src/repositories/workerProjectRepository.ts`

---

15. MAIMLファイル取得（プロジェクト） ✅

エンドポイント: `GET /api/v1/projects/:projectName/maiml`

説明: プロジェクトの最新MAIMLファイルをXML文字列で取得（プロジェクト管理画面・指示確認タブのフロー図表示で使用）

認証: 必須

権限: 全ロール（`authMiddleware` のみ、`checkPermission` なし）

ミドルウェア: `authMiddleware` → コントローラー

重要: 実験作業者を含むすべてのロールが閲覧可能

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名（UUIDではなく名前文字列）

クエリパラメータ: なし（最新版のMAIMLファイルを常に返却）

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "maimlContent": "<?xml version=\"1.0\" encoding=\"UTF-8\"?>..."
  },
  "meta": {
    "timestamp": "2026-02-16T14:40:00Z",
    "requestId": "req_abc123"
  }
}
```

MAIMLファイルが存在しない場合も 200 OK（空文字列を返却）：

```
{
  "success": true,
  "data": {
    "maimlContent": ""
  },
  "meta": {
    "timestamp": "2026-02-16T14:40:00Z",
    "requestId": "req_abc123"
  }
}
```

レスポンスフィールド説明:

フィールド	型	説明
`maimlContent`	string	MAIMLファイルのXML文字列。ファイルが存在しない場合は空文字列 `""`

エラー時:

- 401: 認証エラー
- 404: プロジェクトが存在しない（ `RESOURCE_NOT_FOUND` ）
- 500: システムエラー（ `SYSTEM_DATABASE_ERROR` , `SYSTEM_S3_ERROR` ）

注意: MAIMLファイルが見つからない場合はエラーではなく、 `maimlContent: ""` で 200 OK を返します。

ビジネスロジック

1. `projectName` からプロジェクトを検索（organization_idでテナント分離）
2. プロジェクトが存在しない場合は 404 を返却
3. S3からMAIMLファイルを取得（ `{organizationId}/projects/{projectName}/{projectName}.maiml` ）
4. ファイルが存在しない場合は `maimlContent: ""` で 200 OK を返却
5. ファイルが存在する場合はXML文字列を `maimlContent` に格納して返却

設計方針

- ✅ **プロジェクトのみ対応**（フォーマットは非対応）
- ✅ **最新版のみ取得**（アーカイブ非対応）
- ✅ **S3から直接取得**（file_metadataテーブル不使用）
- ✅ **全ロール閲覧可能**（実験作業者含む）
- ✅ **ファイル未存在時は空文字列で正常レスポンス**

使用箇所

- 実験管理者 - プロジェクト管理画面 - Petri Netフロー図表示（ `ProjectMaimlContext` ）
- 実験作業者 - 指示確認タブ - フロー図表示（ `InstructionConfirmationTab` → `InstructionFlowDialog` → `ViewRenderer` ）

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/projectController.ts` (`getProjectMaiml`)
- **サービス**: `backend/src/services/projectService.ts` (`getProjectMaiml`)

---

17. タスク登録情報取得 ✅

エンドポイント: `GET /api/v1/projects/:projectName/task-registration`

説明: プロジェクトのタスク登録情報を取得。DBにデータがない場合はMAIMLファイルから解析。

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

レスポンス

成功時 (200 OK):


```
{
  "success": true,
  "data": {
    "tasks": [
      {
        "methodId": "method_001",
        "experimentCondition": 0,
        "trialCount": 1,
        "experimentOrder": 1
      },
      {
        "methodId": "method_002",
        "experimentCondition": 1,
        "trialCount": 3,
        "experimentOrder": 2
      }
    ],
    "isCheckoutLocked": false,
    "isMaimlRegistered": true,
    "experimentConditionOptions": [
      {
        "value": 0,
        "label": "全て同一条件"
      },
      {
        "value": 1,
        "label": "実験ごとに条件変更"
      }
    ]
  },
  "meta": {
```

```
    "timestamp": "2026-02-20T09:00:00Z",
    "requestId": "req_abc123"
  }
}
```

レスポンスフィールド説明:

フィールド	型	説明
`tasks`	array	タスク一覧
`tasks[].methodId`	string	メソッドID
`tasks[].experimentCondition`	number	実験条件（code_masterから取得）
`tasks[].trialCount`	number	試行回数（1-999）
`tasks[].experimentOrder`	number	実験順序
`isCheckoutLocked`	boolean	チェックアウト状態（true: 編集不可）
`isMaimlRegistered`	boolean	MAIMLファイル登録状態（false: 保存不可）
`experimentConditionOptions`	array	実験条件の選択肢（code_masterから取得）

エラー時:

- 400: バリデーションエラー
- 401: 認証エラー
- 403: 権限不足
- 404: プロジェクトが存在しない
- 500: システムエラー

ビジネスロジック

取得処理のフロー:

1. プロジェクト存在確認（organization_idでフィルタ）
2. project_taskテーブルからデータ取得
    - データあり → タスク一覧を返却
    - データなし → 3へ
3. MAIMLファイルをS3から取得
4. MAIMLParserTypedでメソッドID一覧を抽出
5. デフォルト値でタスク一覧を生成
    - experimentCondition: 0（全て同一条件）
    - trialCount: 1
    - experimentOrder: 1, 2, 3...
6. code_masterから実験条件オプションを取得
7. チェックアウト状態とMAIML登録状態を取得
8. レスポンスを返却

初期値:

- `experimentCondition`: 0（全て同一条件）
- `trialCount`: 1
- `experimentOrder`: メソッド出現順（1, 2, 3...）

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/taskRegistrationController.ts`（`getTaskRegistration`）
- **サービス**: `backend/src/services/taskRegistrationService.ts`（`getTaskRegistration`）
- **リポジトリ**:
  - `backend/src/repositories/taskRepository.ts`（`findByProject`）
  - `backend/src/repositories/projectRepository.ts`（`findByName`）
  - `backend/src/repositories/codeMasterRepository.ts`（`findByDivision`）
- **ユーティリティ**: `backend/src/utils/maimlParserTyped.ts`（`getMethods`）

---

18. タスク登録保存 ✅

エンドポイント: `POST /api/v1/projects/:projectName/task-registration`

説明: タスク登録情報を保存。project_taskとproject_task_detailテーブルに登録。

認証: 必須

権限: `experiment_admin`, `system_admin`

優先度: ⭐⭐⭐ 高

リクエストパラメータ

パスパラメータ:

パラメータ	型	必須	説明
`projectName`	string	○	プロジェクト名

ボディ（JSON）:

```
{
  "tasks": [
    {
      "methodId": "method_001",
      "experimentCondition": 0,
      "trialCount": 1
    },
    {
      "methodId": "method_002",
      "experimentCondition": 1,
      "trialCount": 3
    }
```

```
  ]
}
```

バリデーション:

フィールド	型	必須	制約
`tasks`	array	○	1個以上
`tasks[].methodId`	string	○	1-255文字
`tasks[].experimentCondition`	number	○	code_masterに存在する値
`tasks[].trialCount`	number	○	1-999

レスポンス

成功時 (200 OK):

```
{
  "success": true,
  "data": {
    "success": true,
    "message": "タスク登録情報を保存しました"
  },
  "meta": {
    "timestamp": "2026-02-20T09:00:00Z",
    "requestId": "req_abc123"
  }
}
```

エラー時:

- 400: バリデーションエラー、試行回数が範囲外
- 401: 認証エラー
- 403: 権限不足、チェックアウト済み
- 404: プロジェクトが存在しない
- 409: MAIMLファイル未登録
- 500: システムエラー

ビジネスロジック

保存処理のフロー:

1. プロジェクト存在確認（organization_idでフィルタ）
2. チェックアウト状態確認
    - checkout_status = 1 → エラー（編集不可）
3. MAIML登録状態確認
    - project_maiml_flg = false → エラー（保存不可）
4. トランザクション開始
5. project_taskテーブルにUPSERT

- organization_id, project_id, method_id が主キー
- experiment_order（配列のインデックス + 1）
- experiment_condition
- attempts_count（trialCount）
- create_user_id, update_user_id（ログインユーザ）

6. project_task_detailテーブルから既存レコード削除
   - organization_id, project_idで削除

7. project_task_detailテーブルにINSERT
   - 試行回数分のレコードを生成
   - task_no: 1から連番
   - task_status: 0（初期状態）

8. トランザクションコミット
9. レスポンスを返却

実装ファイル

- **ルート**: `backend/src/routes/projectRoutes.ts`
- **コントローラー**: `backend/src/controllers/taskRegistrationController.ts`（`saveTaskRegistration`）
- **サービス**: `backend/src/services/taskRegistrationService.ts`（`saveTaskRegistration`）
- **リポジトリ**:
  - `backend/src/repositories/taskRepository.ts`（`upsertTask`, `deleteTaskDetails`, `insertTaskDetail`）
  - `backend/src/repositories/projectRepository.ts`（`findByName`）

🔒 セキュリティ考慮事項

マルチテナント分離の徹底

必須チェック:

- ☐ すべてのDBクエリに `organization_id` フィルタ
- ☐ すべてのS3操作に `org_{organization_prefix}` プレフィックス
- ☐ クロステナントアクセスの防止

実装例:

```
// ✅ 良い例
const projects = await db
  .selectFrom('projects')
  .where('organization_id', '=', organizationId)
```

```
  .where('project_id', '=', projectId)
  .selectAll()
  .execute();

// ❌ 悪い例
const projects = await db
  .selectFrom('projects')
  .where('project_id', '=', projectId)  // organization_idフィルタなし
  .selectAll()
  .execute();
```

権限チェック

ロール別権限:

- `system_admin`: 所属組織内のプロジェクト操作可能
- `experiment_admin`: 所属組織内のプロジェクト操作可能
- `experiment_worker`: 所属組織内のプロジェクト操作可能

S3署名付きURL

ファイルダウンロード:

- 直接URLではなく署名付きURLを返却
- 有効期限: 1時間
- organization_prefixの検証必須

📚 関連ドキュメント

規約

- API設計ポリシー
- バックエンド規約
- セキュリティ規約

設計書

- API一覧（統合版）
- API実装方針
- データベース構造
- S3構造

UI仕様書

- プロジェクト管理画面

📝 変更履歴

バージョン	日付	変更内容	担当者
2.6.0	2026-02-19 18:00	API一覧の競合を修正し実態に即した内容に更新	Cline
		- API一覧テーブルの重複エントリを削除（#7-15の重複を解消）	
		- 実装状況サマリーを修正（カテゴリ分類を実態に合わせて更新）	
		- チェックアウト/チェックインのエンドポイントを修正（:projectId形式に統一）	
		- 総API数: 16個、実装済み: 13個、未実装: 3個	
2.5.0	2026-02-19 17:02	routeファイルの実装状況を反映	Cline
		- projectRoutes.tsの全エンドポイントを確認	
		- 実装済み: 13個（81%）、未実装: 3個	
		- 新規追加: プロジェクト詳細取得、チェックアウト状態取得	
		- 実装ファイル情報を追加	
2.4.0	2026-02-19 15:00	フロントエンド実装状況を反映	Cline
		- ProjectManagement.tsxの実装状況を確認	
		- API一覧テーブルのコメントを詳細化	
		- フロントエンド実装: getProjectFiles(), getProjectTasks(), downloadFolderAsZip()使用確認	
2.3.0	2026-01-26 15:42	プロジェクト管理APIの実装状況を更新	Cline
		- 実装状況: 4→8個に更新（80%完了）	
		- 新規実装確認: ファイル一覧取得、タスク一覧取得	
		- 詳細検索・公開API: ルート定義のみ確認（実装詳細は要確認）	
		- 未実装: 指示送信、指示一覧取得（2個）	
2.2.0	2026-01-26	プロジェクトリロードAPI削除（10エンドポイント）	Cline
		- `/api/v1/projects/:projectName/reload` エンドポイント削除	
		- 理由: 既存のプロジェクト一覧・ファイル一覧・タスク一覧APIで代用可能	
		- フロントエンドは既存APIを組み合わせたリロード処理に変更済み	
		- 総API数: 11→10、実装済み: 5→4 (40%)	
2.0.0	2026-01-21	API一覧に基づき全面改訂、実装状況反映	Cline
1.0.0	2024-12-XX	初版作成	Development Team

API #16: レビュー待ちタスク一括完了

概要

項目	内容
HTTPメソッド	POST
エンドポイント	`/api/v1/projects/:projectName/tasks/complete-review`
説明	指定プロジェクトのレビュー待ち（status=3）タスクを一括で完了状態にする

実装状態	✅ 実装済み
権限	experiment_admin 以上（Manager権限）
呼び出し元	ProjectManagement.tsx（タスク確認完了ボタン）

リクエスト

パスパラメータ

パラメータ	型	必須	説明
projectName	string	✅	プロジェクト名

リクエストボディ

なし

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "message": "レビュー待ちタスクを完了しました",
  "data": {
    "completedCount": 3
  }
}
```

フィールド	型	説明
completedCount	number	完了に変更されたタスク数

エラー時

ステータス	エラーコード	説明
400	INVALID_PROJECT	プロジェクトが存在しない
403	FORBIDDEN	権限不足
404	NOT_FOUND	レビュー待ちタスクが存在しない

API #17: タスク登録状況取得

概要

項目	内容
HTTPメソッド	GET
エンドポイント	`/api/v1/projects/:projectName/instructions/status`
説明	指定プロジェクトの指示登録状況（未送信・送信済み件数など）を取得する
実装状態	✅ 実装済み

権限	experiment_admin 以上（Manager権限）
呼び出し元	InstructionRegistrationTab.tsx

リクエスト

パスパラメータ

パラメータ	型	必須	説明
projectName	string	✅	プロジェクト名

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "draftCount": 2,
    "sentCount": 5,
    "totalCount": 7
  }
}
```

フィールド	型	説明
draftCount	number	未送信（下書き）の指示件数
sentCount	number	送信済みの指示件数
totalCount	number	合計指示件数

API #18: 指示更新

概要

項目	内容
HTTPメソッド	PUT
エンドポイント	`/api/v1/projects/:projectName/instructions/:instructionId`
説明	指定した指示の内容を更新する（未送信の指示のみ更新可能）
実装状態	✅ 実装済み
権限	experiment_admin 以上（Manager権限）
呼び出し元	InstructionRegistrationTab.tsx

リクエスト

パスパラメータ

パラメータ	型	必須	説明

projectName	string	✅	プロジェクト名
instructionId	string	✅	指示ID

リクエストボディ

```
{
  "content": "更新後の指示内容",
  "taskId": "task-001",
  "targetWorkerIds": ["user-uuid-1", "user-uuid-2"]
}
```

フィールド	型	必須	説明
content	string	✅	指示内容
taskId	string	✅	対象タスクID
targetWorkerIds	string[]	✅	送信対象ワーカーのユーザーIDリスト

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "instructionId": "instruction-uuid",
    "content": "更新後の指示内容",
    "taskId": "task-001",
    "status": "draft",
    "updatedAt": "2026-03-06T06:35:00.000Z"
  }
}
```

エラー時

ステータス	エラーコード	説明
400	ALREADY_SENT	送信済み指示は更新不可
403	FORBIDDEN	権限不足
404	NOT_FOUND	指示が存在しない

API #19: 指示削除

概要

項目	内容
HTTPメソッド	DELETE
エンドポイント	`/api/v1/projects/:projectName/instructions/:instructionId`
説明	指定した指示を削除する（未送信の指示のみ削除可能）

実装状態	✅ 実装済み
権限	experiment_admin 以上（Manager権限）
呼び出し元	InstructionRegistrationTab.tsx

リクエスト

パスパラメータ

パラメータ	型	必須	説明
projectName	string	✅	プロジェクト名
instructionId	string	✅	指示ID

リクエストボディ

なし

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "message": "指示を削除しました"
}
```

エラー時

ステータス	エラーコード	説明
400	ALREADY_SENT	送信済み指示は削除不可
403	FORBIDDEN	権限不足
404	NOT_FOUND	指示が存在しない

API #20: 指示一括送信

概要

項目	内容
HTTPメソッド	POST
エンドポイント	`/api/v1/projects/:projectName/instructions/send`
説明	プロジェクトの未送信指示を一括でワーカーに送信する。送信と同時にproject_taskレコードが生成される
実装状態	✅ 実装済み
権限	experiment_admin 以上（Manager権限）
呼び出し元	InstructionRegistrationTab.tsx

リクエスト

パスパラメータ

パラメータ	型	必須	説明
projectName	string	✅	プロジェクト名

リクエストボディ

なし（未送信指示を全件送信する）

レスポンス

成功時（200 OK）

```
{
  "success": true,
  "data": {
    "sentCount": 3,
    "taskIds": ["task-uuid-1", "task-uuid-2", "task-uuid-3"]
  }
}
```

フィールド	型	説明
sentCount	number	送信された指示件数
taskIds	string[]	生成されたタスクIDリスト

エラー時

ステータス	エラーコード	説明
400	NO_DRAFT_INSTRUCTIONS	送信対象の未送信指示がない
403	FORBIDDEN	権限不足

📝 変更履歴（追記分）

バージョン	日付	変更内容	担当者
2.8.0	2026-03-06	ProjectManagement画面から呼び出されるAPI追記	Cline
		- API #16: レビュー待ちタスク一括完了 POST /tasks/complete-review	
		- API #17: タスク登録状況取得 GET /instructions/status	
		- API #18: 指示更新 PUT /instructions/:instructionId	
		- API #19: 指示削除 DELETE /instructions/:instructionId	
		- API #20: 指示一括送信 POST /instructions/send	

API #21: テンプレート実績取得

概要

項目	内容
HTTPメソッド	GET
エンドポイント	`/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data`
説明	指定トランジションに隣接するプレースのテンプレート実績データ（template_data）を取得する
実装状態	✅ 実装済み
権限	`experiment_worker` 以上（機能CD: `050000`）読取可
ミドルウェア	`authMiddleware` → `tenantIsolation` → `checkPermission('050000', 0)` → バリデーション
呼び出し元	DataRecordDialog

リクエスト

パスパラメータ

パラメータ	型	必須	説明
`projectName`	string	✅	プロジェクト名（1〜255文字）
`methodId`	string	✅	メソッドID（1〜255文字）
`taskNo`	integer	✅	タスク番号（1以上の整数）

クエリパラメータ

パラメータ	型	必須	説明
`transitionId`	string	✅	トランジションID（1〜255文字）。このトランジションに隣接するプレースのデータを返す

注意: `transitionId` は必須クエリパラメータです。`API一覧_実験作業者画面.md` には記載がありませんが、実装上必須です。

レスポンス

成功時（200 OK）


```
{
  "success": true,
  "data": {
    "projectName": "sample_project",
    "methodId": "method_001",
    "taskNo": 1,
    "transitionId": "t001",
    "adjacentPlaceIds": ["place_001", "place_002"],
    "templates": [
      {
        "placeId": "place_001",
        "templateType": "material",
        "templateId": "tmpl_mat_001",
        "data": {
          "templateId": "tmpl_mat_001",
          "templateType": "material",
          "properties": [
            {
              "key": "reagentName",
```

```
              "xsiType": "stringType",
              "value": "NaCl",
              "units": null
            },
            {
              "key": "amount",
              "xsiType": "floatType",
              "value": 1.5,
              "units": "g"
            }
          ]
        },
        "recordedBy": "user-uuid",
        "recordedAt": "2026-03-01T10:00:00Z"
      },
      {
        "placeId": "place_002",
        "templateType": "condition",
        "templateId": "tmpl_cond_001",
        "data": {
          "templateId": "tmpl_cond_001",
          "templateType": "condition",
          "properties": [
            {
              "key": "temperature",
              "xsiType": "floatType",
              "value": null,
              "units": "℃"
            }
          ]
        },
        "recordedBy": null,
        "recordedAt": null
      }
    ],
    "totalCount": 2
  },
  "meta": {
    "timestamp": "2026-03-12T10:00:00Z",
    "requestId": "req_abc123"
  }
}
```


レスポンスフィールド説明

フィールド	型	説明
`projectName`	string	プロジェクト名
`methodId`	string	メソッドID
`taskNo`	number	タスク番号
`transitionId`	string	トランジションID
`adjacentPlaceIds`	string[]	トランジションに隣接するプレースIDの配列（入力・出力プレース両方）
`templates`	array	テンプレート実績一覧
`templates[].placeId`	string	プレースID
`templates[].templateType`	string	テンプレート種別（ `material` / `condition` / `result` ）
`templates[].templateId`	string	テンプレートID
`templates[].data`	object	テンプレートデータ（PropertySet形式）

`templates[].data.templateId`	string	テンプレートID
`templates[].data.templateType`	string	テンプレート種別
`templates[].data.properties`	array	プロパティ一覧
`templates[].data.properties[].key`	string	プロパティキー
`templates[].data.properties[].xsiType`	string	データ型（`stringType`, `floatType`, etc.）
`templates[].data.properties[].value`	any	プロパティ値（未記録の場合は `null`）
`templates[].data.properties[].units`	string|null	単位
`templates[].recordedBy`	string|null	記録者のユーザーID（未記録の場合は `null`）
`templates[].recordedAt`	string|null	記録日時（ISO 8601）（未記録の場合は `null`）
`totalCount`	number	テンプレート件数

注意: DBにデータがない場合（未記録）は、MAIMLファイルのテンプレート定義から初期値（`value: null`）のフォールバックデータを返します。

エラー時

ステータス	説明
400	バリデーションエラー（`transitionId` 未指定など）
404	プロジェクト / メソッド / トランジションが見つからない
500	サーバーエラー（S3接続エラー、MAIMLパースエラー等）

ビジネスロジック

1. `projectName` → `projectId` を解決（SQLファイル: `projects/findIdByName.sql`）
2. `template_data` テーブルから対象タスクの全レコードを取得
3. S3/Swift から MaiML ファイルをダウンロード（`{organizationId}/projects/{projectName}/{projectName}.maiml`）
4. `MaimlParserTyped` でペトリネット構造を解析
5. 指定 `methodId` の method を特定し `transitionId` の存在を確認
6. アーク情報から隣接プレースID を特定（入力プレース・出力プレース両方）
7. `template_data` レコードを隣接プレースIDでフィルタリング
8. DBにデータがないプレースは MAIMLテンプレート定義からフォールバック（`value: null`）

実装ファイル

- **ルート**: `backend/src/routes/experimentRoutes.ts`
- **コントローラー**: `backend/src/controllers/experimentController.ts`（`getTemplateData`）
- **サービス**: `backend/src/services/experimentDataService.ts`（`getTemplateData`）
- **リポジトリ**: `backend/src/repositories/templateDataRepository.ts`（`findByTaskNo`）

API #22: 実験データ一括保存

概要

項目	内容
HTTPメソッド	POST
エンドポイント	`/api/v1/experiments/projects/:projectName/methods/:methodId/tasks/:taskNo/data`
説明	複数プレースの実験データを一括保存（upsert）する。みなし進捗が有効な組織ではトランジションステータスを自動更新する
実装状態	✅ 実装済み
権限	`experiment_worker` のみ（機能CD: `050000`）書込可
ミドルウェア	`authMiddleware` → `tenantIsolation` → `checkPermission('050000', 1)` → バリデーション
呼び出し元	DataRecordDialog

注意: `API一覧_実験作業者画面.md` には「1リクエストにつき1テンプレート分」と記載されていますが、実装は**バッチ処理**（`records` 配列で複数件を一括保存）です。

リクエスト

パスパラメータ

パラメータ	型	必須	説明
`projectName`	string	✅	プロジェクト名（1〜255文字）
`methodId`	string	✅	メソッドID（1〜255文字）
`taskNo`	integer	✅	タスク番号（1以上の整数）

リクエストボディ

```
{
  "records": [
    {
      "placeId": "place_001",
      "data": {
        "templateId": "tmpl_mat_001",
        "templateType": "material",
        "properties": [
          {
            "key": "reagentName",
            "xsiType": "stringType",
            "value": "NaCl",
            "units": null
          },
          {
            "key": "amount",
            "xsiType": "floatType",
            "value": 1.5,
            "units": "g"
          }
        ]
      },
      "comment": "記録メモ（省略可）"
    }
  ]
}
```

バリデーション

フィールド	型	必須	制約
`records`	array	✅	1件以上50件以下
`records[].placeId`	string	✅	1〜255文字
`records[].data`	object	✅	オブジェクト形式
`records[].data.templateId`	string	✅	1〜255文字
`records[].data.templateType`	string	✅	`material` / `condition` / `result` のいずれか
`records[].data.properties`	array	✅	配列形式
`records[].comment`	string	-	最大2000文字

レスポンス

成功時（**201 Created**）

```
{
  "success": true,
  "data": {
    "saved": 2,
    "records": [
      {
        "organization_id": "org-uuid",
        "project_id": "project-uuid",
        "method_id": "method_001",
        "task_no": 1,
        "place_id": "place_001",
        "property_set": {
          "templateId": "tmpl_mat_001",
          "templateType": "material",
          "properties": [...]
        },
        "create_user_id": "user-uuid",
        "update_user_id": "user-uuid",
        "create_at": "2026-03-12T10:00:00Z",
        "update_at": "2026-03-12T10:00:00Z"
      }
    ]
  },
  "meta": {
    "timestamp": "2026-03-12T10:00:00Z",
    "requestId": "req_abc123"
  }
}
```

レスポンスフィールド説明

フィールド	型	説明
`saved`	number	保存されたレコード件数
`records`	array	保存されたレコードの配列（`template_data` テーブルのDBレコード形式）

エラー時

ステータス	説明

400	バリデーションエラー
403	プロジェクトが編集不可（`project_status ≠ 2`）
404	タスクが見つからない
500	サーバーエラー

ビジネスロジック

1. `projectName` → `projectId` + `projectStatus` を解決（SQLファイル: `projects/findByNameWithStatus.sql`）
2. `project_status = 2`（タスク実行中）のみ編集可能。それ以外は 403 エラー
3. タスク存在確認（SQLファイル: `tasks/checkExists.sql`）
4. トランザクション開始
5. `records` の各要素を `template_data` テーブルに UPSERT（キー: `organization_id`, `project_id`, `method_id`, `task_no`, `place_id`）
6. `use_deemed_progress = true` の組織の場合、アーク情報からトランジションIDを解決し `transition_data` の `status` / `progress_rate` を自動更新（みなし進捗）
   - 実績登録済み数 < 総プレース数 → IN_PROGRESS (3)
   - 実績登録済み数 === 総プレース数 → COMPLETED (5)
7. トランザクションコミット

実装ファイル

- **ルート**: `backend/src/routes/experimentRoutes.ts`
- **コントローラー**: `backend/src/controllers/experimentController.ts`（`saveBatchData`）
- **サービス**: `backend/src/services/experimentDataService.ts`（`saveBatchData`）
- **リポジトリ**: `backend/src/repositories/templateDataRepository.ts`（`upsert`）

API #23: トランジションステータス更新（廃止済み）

概要

項目	内容
HTTPメソッド	PUT
エンドポイント	`/api/v1/experiments/projects/:projectId/methods/:methodId/transitions/:transitionId/status`
説明	**このエンドポイントは廃止済みです（410 Gone）。** 新しいエンドポイントを使用してください
実装状態	⛔ 廃止済み（410 Gone）
廃止日	2026-02-18
移動先	`PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status`
移動先仕様書	`docs/design/api/api_progress_monitoring.md` - API #6

廃止の背景

2026-02-18 にトランジションステータス更新APIが `experimentRoutes.ts` から `progressRoutes.ts` へ移動されました。

移行内容：

- **旧エンドポイント**: `PUT /api/v1/experiments/projects/:projectId/methods/:methodId/transitions/:transitionId/status`
  - パスパラメータに `projectId`（UUID）を使用
- **新エンドポイント**: `PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status`
  - パスパラメータに `projectName`（文字列）を使用
  - HTTPメソッドが `PUT` → `PATCH` に変更

現在の動作（廃止済みエンドポイント）

旧エンドポイントへのリクエストは **410 Gone** を返します：

```
{
  "success": false,
  "error": {
    "code": "ENDPOINT_DEPRECATED",
    "message": "このエンドポイントは廃止されました。新しいエンドポイントを使用してください。",
    "details": {
      "deprecated_endpoint": "PUT /api/v1/experiments/projects/:projectId/methods/:methodId/transitions/:transitionId/status",
      "new_endpoint": "PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status",
      "migration_guide": "docs/20260218_1730_トランジションステータス更新API移動_PLAN.md"
    }
  },
  "meta": {
    "timestamp": "2026-03-12T10:00:00Z",
    "requestId": "req_abc123"
  }
}
```

移行先API

正式なトランジションステータス更新APIは `api_progress_monitoring.md` の API #6 を参照してください。

移行先エンドポイント: `PATCH /api/v1/progress/projects/:projectName/methods/:methodId/transitions/:transitionId/status`

`API一覧_実験作業者画面.md` の記載について: 同ドキュメントには `PUT /api/v1/experiments/.../status` として記載されていますが、これは古い情報です。実際の実装は `PATCH /api/v1/progress/.../status` です。

実装ファイル（廃止済み処理）

- **ルート**: `backend/src/routes/experimentRoutes.ts`（廃止済みルートは定義なし）

- **コントローラー**: `backend/src/controllers/experimentController.ts`（`updateTransitionStatus`）
  ※410を返すスタブ

📝 変更履歴（3.0.0）

バージョン	日付	変更内容	担当者
3.0.0	2026-03-12	実験データ関連API（#21〜#23）をソースコード実装に基づき追加	Cline
		- ヘッダー更新: 総API数 20個、実装済み 18個、未実装 1個、廃止済み 1個	
		- 実装状況サマリーに「実験データ」カテゴリを追加	
		- API一覧テーブルに #21, #22, #23 を追加	
		- API #21: テンプレート実績取得 GET /experiments/.../data を追加（`API一覧_実験作業者画面.md` との差異: エンドポイント末尾は `/data`、`transitionId` クエリパラメータが必須、レスポンスはネスト構造）	
		- API #22: 実験データ一括保存 POST /experiments/.../data を追加（`API一覧_実験作業者画面.md` との差異: バッチ処理、`records` 配列形式、201 Created）	
		- API #23: トランジションステータス更新（廃止済み）を追加（旧 PUT エンドポイントは 410 Gone、新エンドポイントは PATCH /progress/.../status）	

📝 変更履歴（2.9.0）

バージョン	日付	変更内容	担当者
2.9.0	2026-03-12	実験作業者画面のプロジェクト関連APIをソースコード実装に基づき修正	Cline
		- ヘッダー更新: 総API数 17個、実装済み 16個、未実装 1個、完了率 94%	
		- 実装状況サマリー更新（合計 16/17 → 完了率 94%）	
		- API一覧テーブル #13 エンドポイントを `:projectId` → `:projectName` に修正	
		- API #13 MAIMLファイル取得: ✅実装済みに変更、パスパラメータを `projectName`（文字列）に修正、権限を全ロールに変更、レスポンスを `{ maimlContent: string }` に修正、ファイル未存在時は空文字列で 200 OK を返す仕様に修正、実装ファイル追記	
		- API #13 承認依頼: ✅実装済みに変更、ビジネスロジックをソースコード実装に合わせて修正（タスク完了チェック・問題チェックは実装なし）、レスポンスから `notification_id` を削除、エラーコードを実装に合わせて修正、ミドルウェア情報追記	
		- API #14 実験作業者用プロジェクト一覧: ✅実装済みに変更、`status` パラメータを数値文字列 `"1"/"2"/"3"` に修正、レスポンスをcamelCase形式（`projectId`, `projectName`, `projectStatus`, `useDeemedProgress`, `createdAt`, `updatedAt`）に修正、実装ファイル追記	

作成者: Cline **作成日**: 2026-01-21 16:30 **最終更新**: 2026-03-12 13:21